

Что изменилось в бэкенде

Отчёт для того, кто видел код до этих правок: что появилось, как устроено внутри и почему сделано именно так.

78 тестов 1 миграция 14 таблиц 3 механизма

СОДЕРЖАНИЕ

01	Словарь	1
02	Проекция пользователей: что было и что стало	2
03	Новые таблицы	4
04	Эндпоинты	6
05	Инварианты в базе	7
06	Механизм 1: сессии	8
07	Механизм 2: блокировки строк	11
08	Механизм 3: сколько заявок тянет менеджер	12
09	Миграции	13
10	Заведение администратора	14

Словарь

Понятия, которые дальше используются без пояснений.

Воркфлоу — маршрут, по которому движется работа с вузом. В ТЗ он описан как последовательность из 14 шагов: от поиска контактного лица до повышения квалификации преподавателей. Хранится в таблице `workflows`.

Стадия (`stages`) — один шаг этого маршрута: «Встреча», «Подписание договора», «Передача материалов». У стадии есть позиция в маршруте и два флага, которые определяют, как она влияет на нагрузку менеджера (см. ниже). Стадии принадлежат конкретному воркфлоу.

Переход (`workflow_transitions`) — разрешённое ребро между стадиями: откуда куда можно двигаться. Переход с пустым `from_stage_id` означает вход в маршрут.

Заявка, она же взаимодействие (`interactions`) — конкретная работа с конкретным вузом по конкретному продукту. Едет по стадиям воркфлоу, имеет ответственного менеджера, может вставать на паузу.

Роли. *Менеджер* ведёт заявки. *Руководитель* распределяет менеджеров и следит за ними. *Администратор* занимается справочниками, правами и настройками, но бизнес-действий не совершает — не двигает заявки и не согласует переходы.

Слот, ёмкость — у менеджера есть потолок одновременных заявок (по умолчанию 5). Каждая заявка, находящаяся в работе, занимает один слот. Сколько именно слотов занято — считается по правилу, описанному в разделе про ёмкость.

Проекция пользователей — локальная копия учётных записей Keycloak в нашей таблице `users`. Keycloak остаётся источником правды о том, кто существует и какая у него роль; наша база — о том, чем он владеет и насколько загружен.

Инвариант — правило, записанное в самой базе (внешний ключ, ЧЕКС, триггер, уникальный индекс). Его нельзя обойти ни из кода, ни руками через `psql`.

Блокировка строки — `SELECT ... FOR UPDATE`: «беру эту строку себе до конца транзакции, остальные ждут». Нужна, чтобы две параллельные операции не затёрли друг друга.

Проекция пользователей: что было и что стало

Таблица `users` с тремя подтипами существовала и раньше. Но проекцией в рабочем смысле она не была, по трём причинам.

Логин и почта не были колонками. В модели стояло:

```
__allow_unmapped__ = True
username: str | None = None      # обычный атрибут класса,
email: str | None = None        # а не mapped_column
```

Эти поля жили только в памяти, и чтобы их заполнить, `UserRepository.get()` ходил в Keycloak по HTTP. А `get_current_user` вызывал именно его — значит, сетевой поход случался на каждом запросе к API.

Таблица лежала в отдельном наборе метаданных. `User` наследовался от `RawBase`, всё остальное — от `Base`. Для SQLAlchemy это два несвязанных набора таблиц, и внешний ключ из `interactions` на `managers` поставить было нельзя. То есть главное, ради чего нужна локальная копия, не работало.

Строки подтипов не создавались. Эндпоинт создания пользователя конструировал голый `User(...)`. При наследовании «одна таблица на класс» это кладёт строку в `users` и не создаёт парную строку в `managers`. Дальше такой объект падает при первом же обращении к полям подтипа.

Что сделали

	БЫЛО	СТАЛО
<code>username</code> , <code>email</code>	атрибуты в памяти, из Keycloak на каждый запрос	колонки <code>String(255)</code> с индексом
Базовый класс	<code>RawBase</code> , отдельные метаданные	общий <code>Base</code> — внешние ключи заработали
Создание	голый <code>User</code> , подтип не появлялся	<code>user_class_for_role(role)</code> + проверка в репозитории
Активность	отличить уволенного было нечем	<code>is_active</code> , по умолчанию <code>false</code>
Расхождение с Keycloak	не отслеживалось	<code>identity_sync_status</code> , <code>role_transition_status</code>
Нагрузка	полей не было	<code>max_active_projects</code> , <code>manual_workload_status</code> , <code>max_subordinates</code>
Удаление подтипов	<code>ON DELETE CASCADE</code>	<code>ON DELETE RESTRICT</code>
<code>managers.supervisor_id</code>	<code>SET NULL</code>	<code>RESTRICT</code>
Хранение роли	<code>MANAGER</code>	<code>manager</code>
Время изменения	не было	<code>created_at</code> , <code>updated_at</code> + триггер

Про регистр роли. `Mapped[UserRole]` по умолчанию пишет в базу имя члена перечисления, то есть `MANAGER`. Весь SQL оргструктуры сравнивает роль со строкой `'manager'`, и такие запросы молча возвращали бы пусто. Лечится параметром при объявлении колонки:

```
role: Mapped[UserRole] = mapped_column(
    SAEnum(UserRole, name="userrole",
        values_callable=lambda enum: [member.value for member in enum]),
    index=True,
)
```

Про `user_class_for_role`. Функция возвращает конкретный класс по роли, а репозиторий при создании дополнительно проверяет, что переданный объект этому классу соответствует. Две проверки на одно правило — потому что нарушение тихое: строка ляжет в базу, а сломается всё гораздо позже.

Новые таблицы

`sessions` — переписана

Было четыре колонки: `id`, `user_id`, `access_token`, `refresh_token`, всё открытым текстом.

Стало:

КОЛОНКА	ТИП	ЗАЧЕМ
<code>id</code>	<code>String(64)</code>	<code>sha256</code> от ключа, выданного клиенту
<code>user_id</code>	<code>String(255)</code> , индекс	Сессии массово отзываются по пользователю
<code>refresh_token</code>	<code>Text</code>	Зашифрован, см. раздел про сессии
<code>created_at</code>	<code>timestampz</code>	
<code>expires_at</code>	<code>timestampz</code> , индекс	Срок жизни, берётся из ответа Keycloak
<code>last_validated_at</code>	<code>timestampz</code>	Когда последний раз сверялись с Keycloak

Внешнего ключа на `users` намеренно нет: сессия создаётся сразу после входа, а локальная строка пользователя может появиться позже — её заводит фоновая сверка.

`audit_logs` — новая

Журнал действий: кто, что, над кем, когда, что было и что стало. Нужен по требованиям безопасности из ТЗ.

КОЛОНКА	ЗАЧЕМ
<code>actor_type</code>	<code>USER</code> или <code>SYSTEM</code> — человек или фоновый процесс
<code>actor_id</code>	Кто сделал; у системных записей всегда пусто
<code>target_type</code> , <code>target_id</code>	Над кем или над чем

КОЛОНКА	ЗАЧЕМ
<code>event_type</code>	Что именно произошло, 24 значения
<code>old_value</code> , <code>new_value</code>	<code>JSONB</code> — состояние до и после
<code>created_at</code>	Индексируется, по нему ищут

Колонки `updated_at` нет и не будет: записи не меняются. Это не договорённость, а триггер в базе — любой `UPDATE`, `DELETE` или `TRUNCATE` завершается ошибкой.

Сейчас в неё ещё никто не пишет: писатели появятся вместе с операциями оргструктуры.

`pending_org_actions` — новая

Очередь дел, которые нельзя сделать прямо сейчас.

Зачем очередь. Сотрудника уволили в Keycloak, но за ним висят три заявки. Удалить его нельзя — заявки осиротеют. Оставить как есть тоже нельзя. Значит, кладём задачу в очередь, и фоновый процесс будет к ней возвращаться, пока руководитель не разберёт заявки. Очередь именно в базе, а не в памяти: между постановкой задачи и её выполнением могут пройти недели и несколько перезапусков приложения.

Три типа задач: `OFFBOARDING_MANAGER` (уволился менеджер), `OFFBOARDING_SUPERVISER` (уволился руководитель, надо пристроить его подчинённых), `ROLE_TRANSITION` (сменилась роль, надо перенести человека в другую таблицу подтипа).

Задачу будет брать фоновый процесс, и чтобы два процесса не взялись за одну, есть аренда:

КОЛОНКА	ЗАЧЕМ
<code>worker_id</code>	Кто взял задачу
<code>lease_until</code>	До какого момента она за ним закреплена
<code>lease_version</code>	Номер поколения аренды, растёт при каждом захвате
<code>retry_count</code> , <code>max_attempts</code>	Счётчик попыток при технических сбоях
<code>next_retry_at</code>	Когда пробовать снова
<code>last_error</code>	Текст последней ошибки

Зачем `lease_version`. Процесс может зависнуть — например, машину усыпили. Он очнётся через час, когда задачу давно перехватил и выполнил другой. Если бы он просто записал результат, то затёр бы чужую работу. Поэтому при записи он обязан предъявить тот номер поколения, с которым брал задачу; за это время номер уже вырос, и запись не пройдёт.

Что добавилось к существующим таблицам

`interactions` :

- `owner_id` теперь ссылается на `managers.id`, а не просто хранит строку. Назначить владельцем заявки руководителя или администратора физически невозможно.
- `last_owner_id` → `users.id` — кто владел раньше. Когда менеджера переводят на другую должность, строка из `managers` исчезает, а авторство должно остаться.
- `is_paused`, `pause_state`, `paused_until` — состояние паузы.

Четыре состояния паузы. `ACTIVE` — работа идёт. `PAUSED_MANUAL` — остановлена без срока (вуз попросил подождать). `PAUSED_TIMED` — остановлена до конкретной даты. `EXPIRED_WAITING_CAPACITY` — срок вышел, но вернуть заявку в работу нельзя: у менеджера нет свободного слота. Последнее состояние нужно, чтобы заявка не потерялась и её было видно в отчётах.

`stages` — два флага, которые и определяют влияние стадии на нагрузку:

- `is_terminal` — стадия завершает работу (договор закрыт, либо отказались);
- `consumes_capacity` — стадия занимает слот менеджера.

Почему два флага, а не один. Есть стадии, которые формально в работе, но внимания почти не требуют — например, ожидание ответа от вуза. Их можно пометить как не занимающие слот, и менеджер сможет взять ещё одну заявку. Терминальная стадия слот не занимает по определению, поэтому сочетание «завершена и занимает» запрещено ограничением в базе.

`workflows` — `is_published`. Пока маршрут не опубликован, стадии можно менять; после публикации структура фиксируется, потому что по ней уже едут живые заявки.

Эндпоинты

МАРШРУТ

ЧТО ДЕЛАЕТ

`GET /auth/`

Редирект на форму входа Keycloak

МАРШРУТ	ЧТО ДЕЛАЕТ
GET /auth/callback	Обмен кода на токены, создание сессии, установка куки
POST /auth/logout	Выход из Keycloak и удаление сессии
GET /api/v1/users/me	Текущий пользователь
GET /api/v1/users/	Список (админ)
GET /api/v1/users/{id}	Один пользователь (админ)
POST /api/v1/users/	Завести пользователя (админ)
PATCH /api/v1/users/{id}	Изменить (админ)
DELETE /api/v1/users/{id}	Удалить (админ)
POST /api/v1/users/connections/{supervisor_id}?manager_id=...	Привязать менеджера к руководителю (админ)

В /auth остался только вход и выход, управление пользователями переехало в общий /api/v1. Удалён GET /front : он отдавал файл по захардкоженному пути с чужой машины.

Остальные 37 эндпоинтов (/api/v1/workflows , /stages , /transitions , /interactions , /universities , /vendors , /attachments) пока открыты без авторизации. Механизм готов, но не навешан.

Инварианты в базе

ЧТО ЗАПРЕЩЕНО	ЧЕМ	ИМЯ
Сделать владельцем заявки не менеджера	Внешний ключ на managers	fk_interactions_owner_id_managers
Удалить менеджера с заявками	ON DELETE RESTRICT	там же
Поставить заявку на стадию чужого воркфлоу	Составной внешний ключ	fk_interactions_state_id_workflow_id_stages
Указать стадию, не указав воркфлоу	CHECK	chk_interaction_stage_requires_workflow
Рассогласовать поля паузы	CHECK на 4 комбинации	chk_interaction_pause_state
Завершающая стадия занимает слот	CHECK	chk_stage_terminal_no_capacity

ЧТО ЗАПРЕЩЕНО	ЧЕМ	ИМЯ
Системная запись журнала с автором	CHECK	chk_audit_actor_invariants
Изменить или удалить запись журнала	Триггер	trg_audit_logs_append_only_guard
Устаревшая метка времени при прямом UPDATE	Триггер	trg_users_updated_at
Две активные задачи одного типа на человека	Частичный уникальный индекс	uq_active_pending_org_action

Два места заслуживают пояснения.

Составной ключ на стадию. У заявки есть и `workflow_id`, и `state_id`, и ничто не мешало поставить заявку воркфлоу А на стадию из воркфлоу Б. Чтобы это запретить, на `stages` добавлена уникальность пары `(id, workflow_id)` — она дублирует первичный ключ, но нужна как цель ссылки, — а `interactions` ссылается на эту пару целиком. Тонкость: составной ключ не проверяется, если хотя бы одна колонка пуста. Поэтому рядом стоит CHECK: стадия без воркфлоу недопустима. А вот воркфлоу без стадии — нормальный случай, это черновик, которому уже выбрали маршрут.

Частичный уникальный индекс. Задачи из очереди не удаляются, они переходят в `COMPLETED` или `CANCELLED` и лежат вечно. Обычная уникальность по паре «тип, пользователь» запретила бы повторный цикл: человека уволили, потом взяли обратно, потом снова уволили. Поэтому индекс сужен условием `WHERE status IN ('PENDING', 'IN_PROGRESS')` — уникальны только активные задачи.

Все инварианты покрыты тестами: в `tests/test_schema_invariants.py` видно, как каждый срабатывает.

Механизм 1: сессии

Вход

1. Пользователь идёт на `/auth/`, его перекидывает в Keycloak.
2. Keycloak возвращает его на `/auth/callback` с кодом.
3. Меняем код на токены.
4. Генерируем случайный ключ: `secrets.token_urlsafe(64)`. Клиенту в куку уходит сам ключ, в базу пишется его `sha256`.
5. Refresh-токен шифруем и кладём в базу.
6. Access-токен не сохраняем вовсе.

Почему в базе хеш. Если база утечёт, по хешам чужие сессии не восстановить — тот же приём, что с паролями. Ключ на 64 байта перебрать нереально, а сравнение по хешу ничем не медленнее.

Почему access-токен не нужен. Он живёт пять минут. Роль и статус мы берём из своей таблицы `users`, а не из токена, поэтому хранить его незачем. Refresh-токен, наоборот, нужен: им выходят из Keycloak и им же сверяются.

Кука ставится с флагами `httponly`, `samesite=lax`, `path=/`, сроком по времени жизни сессии и `secure` из настройки `SESSION_COOKIE_SECURE` — её выключают только для локальной разработки по http.

Шифрование

Refresh-токен шифруется Fernet из библиотеки `cryptography`, ключ берётся из `SESSION_SECRET_KEY`. Можно указать несколько ключей через запятую: шифрование идёт первым, расшифровка — любым. Это даёт смену ключа без разлогаина всех разом.

Fernet выбран вместо голого AES-GCM намеренно: последний требует вручную управлять одноразовым числом, а ошибка в этом месте обесценивает шифрование целиком.

Если расшифровать не удалось — ключ потеряли или сменили без переходного периода — сессия закрывается, и человек просто входит заново. Не 500 на каждом запросе.

Проверка на каждом запросе

Сессия ищется одним запросом:

```
SELECT ... FROM sessions WHERE id = :xеш AND expires_at > now()
```

Срок проверяет сама база. Это не экономия строчки: при отдельной проверке в коде между «проверили» и «использовали» остаётся промежуток, в который сессия может истечь.

Дальше смотрим на `last_validated_at`:

прошло	что делаем
меньше 300 с	В Keycloak не ходим
больше 300 с	Идём сверяться
больше 900 с без успешной сверки	Закрываем сессию

Оба порога настраиваются: `SESSION_REVALIDATE_SECONDS` и `SESSION_REVALIDATE_HARD_LIMIT_SECONDS`.

Сверка

Сверка — это попытка обновить токен через Keycloak. Она отвечает сразу на три вопроса: жива ли сессия в Keycloak, не заблокировали ли человека, не сменили ли ему роль. Если Keycloak обновил токен — всё в порядке.

ИСХОД	ЧТО ДЕЛАЕМ
200	Сохраняем новый refresh-токен, продлеваем срок сессии
4xx	Сессию отозвали или человека заблокировали — удаляем сессию, 401
Сеть не ответила за 3 с	Сессию не трогаем, но возвращаем метку времени назад

Последняя строка важна. Захват сверки сдвигает `last_validated_at` вперёд ещё до похода в сеть. Если бы при сбое мы этот сдвиг оставили, получилось бы, что сверка «прошла», и отсчёт до жёсткого лимита обнулится — сессия пережила бы упавший Keycloak навсегда. Поэтому метка возвращается на прежнее значение, и окно в 900 секунд продолжает идти.

Заодно из ответа достаются `claims` и сверяется роль с локальной проекцией. Расхождение пока только пишется в лог: решения принимаются по базе, а не по токену.

Почему не на каждом запросе

Тогда каждый клик уходил бы в Keycloak, а его падение останавливало бы работу целиком. Раз в пять минут — компромисс: заблокированный сотрудник теряет доступ максимум через пять минут.

Параллельные запросы

Браузер на одном экране шлёт пять-десять запросов разом. Если все они увидят «пора сверяться», то все пойдут обновлять токен. А если в Keycloak включён одноразовый refresh-токен, каждый следующий обнулит предыдущий, и сессия умрёт на ровном месте.

Поэтому перед походом в сеть запрос пытается застолбить сверку:

```
UPDATE sessions SET last_validated_at = now( )
WHERE id = :xesh AND last_validated_at = :to_что_я_видел
RETURNING id
```

Строка обновится только у одного — он и пойдёт в Keycloak. Остальные работают как есть: их данные всё равно свежие в пределах интервала.

Почему хранилище сессий работает на своих транзакциях. Захват должен стать видимым соседним запросам немедленно. Если бы он висел в незакоммиченной транзакции запроса, соседи его бы просто не увидели, и вся затея не работала бы. Поэтому `SessionStore` открывает короткую транзакцию на каждую операцию, независимо от транзакции запроса. Побочный плюс: отзыв сессии не откатится вместе с какой-нибудь бизнес-ошибкой.

Сроки

Срок сессии берётся из поля `refresh_expires_in` ответа Keycloak и продлевается при каждой сверке. Это и есть таймаут простоя, заданный политикой провайдера, — свой параллельный таймер заводить не пришлось.

Считаем по относительным значениям, а не по абсолютной дате внутри токена: так расхождение часов между нашим сервером и Keycloak ни на что не влияет. Плюс тридцать секунд запаса.

Механизм 2: блокировки строк

Задача

Два руководителя одновременно назначают заявки одному менеджеру, у которого свободно одно место. Оба прочитали «занято 4 из 5», оба решили, что место есть, оба записали — стало 6.

Лечится блокировкой: первый берёт строку менеджера себе, второй ждёт, дожидается, видит уже 5 из 5 и получает отказ.

Порядок

Если одна операция берёт сначала строку А, потом Б, а другая — сначала Б, потом А, они встанут намертво: каждая ждёт ту, которую держит другая. База такое замечает и убивает одну из транзакций.

Лечится единым порядком для всех:

```
Supervisor → Manager → User → Interaction → PendingOrgAction → Workflow → Stage
```

А если на одном уровне несколько строк — по возрастанию `id`.

Ловушка

Казалось бы, достаточно написать `select(Manager).with_for_update()`. Но `Manager` — наследник `User`, и такой запрос разворачивается в:

```
SELECT ... FROM users JOIN managers ON users.id = managers.id ... FOR UPDATE
```

Блокируются **обе** строки, причём `users` раньше `managers` — ровно наоборот установленному порядку. Код, написанный «в лоб», ломал бы правило, ради которого всё затевалось.

Решение

Модуль `core/locking.py` с двумя функциями:

```
async def lock_row(session, model, ident):
    stmt = (
        select(model)
        .where(model.id == ident)
        .with_for_update(of=model.__table__) # ключевое место
    )
    return (await session.execute(stmt)).scalar_one_or_none()
```

`FOR UPDATE OF <таблица>` блокирует строки только названной таблицы. Запрос по-прежнему делает `JOIN`, но под замок попадает ровно одна нужная строка.

`lock_rows` берёт несколько строк одного уровня **отдельными запросами** по возрастанию `id`. Не одним запросом с `IN (...) ORDER BY id`, потому что тогда порядок захвата определял бы планировщик базы, а не наш код. Ради двух-трёх строк это допустимая цена.

Любой `FOR UPDATE` в проекте должен идти через эти функции. Тесты в `tests/test_locking.py` проверяют в том числе, что блокировка `managers` действительно не задевает `users`: одна транзакция держит строку менеджера, вторая в это же время свободно берёт строку пользователя.

Механизм 3: сколько заявок тянет менеджер

Правило

Функция `counts_toward_capacity(stage, is_paused)`. Слот занимается, только если выполнено всё сразу:

- стадия у заявки задана — черновик без стадии места не занимает;
- стадия не завершающая;
- у стадии стоит `consumes_capacity`;
- заявка не на паузе.

Три счётчика

Их легко перепутать, а считают они разное:

ФУНКЦИЯ	ЧТО СЧИТАЕТ	ГДЕ НУЖНА
<code>count_capacity_projects</code>	Занятые слоты	Проверка перед назначением
<code>count_open_projects</code>	Незакрытые заявки, включая паузы	Отчёты, витрины
<code>count_owned_nonterminal_interactions</code>	То же плюс черновики без стадии	Можно ли отпустить менеджера

Третий — самый строгий. Пока он не ноль, менеджера нельзя открепить от руководителя и нельзя завершить смену его роли: иначе заявки останутся без ответственного.

Правило описано один раз и используется дважды: как функция на Python для доменной логики и как SQL-выражение для выборки. Второе — чтобы не вытаскивать все заявки в память ради подсчёта.

Страж

Перед тем как повесить на менеджера ещё одну заявку, вызывается

```
validate_capacity_transition(session, manager, delta_slots):
```

ПРОВЕРКА	ОТКАЗ
<code>delta_slots</code> больше нуля?	Если нет — пропускаем без проверок
Сотрудник активен?	400
Готов брать новое (<code>manual_workload_status</code>)?	400
Хватает свободных слотов?	409

Первая строка не формальность. Если нагрузка не растёт — например, заявку наоборот закрывают, — проверки пропускаются. Иначе недоступный менеджер не смог бы закрыть собственную заявку.

Страж имеет смысл только если строка менеджера уже заблокирована: иначе между подсчётом и записью влезет кто-то ещё. Это записано в его документации, а сам захват оставлен вызывающему коду — потому что порядок блокировок важен именно там, где он берётся.

Миграции

Схема больше не создаётся при старте приложения — за неё отвечает Alembic. Сейчас одна миграция, создающая всё сразу.

Автогенерация не видит триггеры, поэтому их блок дописан в конце файла руками. Там же лежит удаление типа перечисления `userrole: drop_table` его не снимает, и без этого повторный `upgrade` падает.

Продакшена нет, базу можно пересоздавать, поэтому цепочку ревизий не копим. При изменении моделей: `alembic downgrade base`, удалить файл ревизии, сгенерировать заново, перенести блок триггеров.

Именованное ограничение задано конвенцией на уровне метаданных — без неё автогенерация выдавала безымянные `drop_constraint`, которые просто не выполняются.

Запуск с нуля:

```
uv sync --all-groups
docker compose up -d
uv run alembic upgrade head
uv run uvicorn quoll.main:app --reload
```

Тесты поднимают отдельную базу `quoll_test_db` и накатывают на неё те же миграции, так что миграция проверяется на каждом прогоне.

Заведение администратора

Раньше администратор приходил из импорта realm Keycloak. Теперь его создаёт приложение при старте (`auth/bootstrap.py`). Две ветки:

- учётки в Keycloak нет — создаём её, идентификатор выдаёт Keycloak, по нему заводим локальную строку;
- учётка есть, а локальной строки нет — например, пересоздали базу — находим по логину и восстанавливаем строку с тем же идентификатором.

Вторая ветка существует потому, что без неё после пересоздания базы администратор не мог бы войти: в Keycloak он есть, в проекции его нет, и `get_current_user` отдаёт 401.

Если Keycloak недоступен, пишем ошибку в лог, но приложение всё равно стартует: иначе порядок запуска контейнеров превращался бы в лотерею.